

CAHIER DES CHARGES

PROJET N°12 — Système de Gestion des Tontines

Plateforme numérique d'épargne solidaire et de finance
participative

Type de document	Cahier des charges fonctionnel & technique
Version	1.0
Domaine	Épargne solidaire / FinTech

1. INTRODUCTION

1.1 Contexte et Présentation du Projet

La tontine est une pratique d'épargne collective profondément enracinée dans les cultures africaines, caribéennes et asiatiques. Elle repose sur la confiance mutuelle entre un groupe de personnes qui s'engagent à cotiser régulièrement une somme fixe, afin que chaque membre puisse, à son tour, bénéficier de la totalité de la cagnotte. Ce mécanisme simple permet un accès rapide à une somme importante sans recours au crédit bancaire traditionnel.

Cependant, la gestion manuelle des tontines souffre de nombreuses limitations : risque de litiges liés au manque de traçabilité, difficulté à coordonner les membres à distance, absence de rappels automatiques pour les cotisations, et manque de transparence dans la gestion des fonds. Ces problèmes entraînent souvent des conflits, des pertes de confiance et l'échec de la tontine.

Le projet "Système de Gestion des Tontines" vise à pallier ces insuffisances en proposant une plateforme numérique complète, sécurisée et transparente qui automatise et digitalise l'ensemble du processus tontinier.

1.2 Rappel du Fonctionnement d'une Tontine

Exemple Concret — Tontine de 5 personnes a 200 000 FCFA/mois

Mois 1 : Personne A recoit 1 000 000 FCFA (les 5 membres ont cotise 200 000 FCFA chacun)

Mois 2 : Personne B recoit 1 000 000 FCFA

Mois 3 : Personne C recoit 1 000 000 FCFA

Mois 4 : Personne D recoit 1 000 000 FCFA

Mois 5 : Personne E recoit 1 000 000 FCFA

=> Chaque membre aura verse 1 000 000 FCFA et reçu 1 000 000 FCFA, mais a des moments differents.

Regle cle : $\text{NombreMembres} = \text{DureeTotale} / \text{FrequenceCotisation}$ (ex: 1 semaine / 1 jour = 7 membres)

1.3 Objectifs du Projet

- Simplifier et automatiser la gestion des contributions et des distributions de fonds.
- Assurer la transparence et l'equite dans la gestion des fonds grace a un systeme de tracabilite complete.
- Faciliter la communication et la coordination entre les membres grace a un systeme de messagerie integre (WebSocket).
- Securiser les transactions financieres et garantir l'identite des participants (KYC).
- Offrir un acces en temps reel aux informations financieres (soldes, historique, prochaines distributions).
- Fournir un cadre legal numerique via la signature electronique d'un contrat de tontine.

Avantages couverts par le systeme

Acces rapide a une grande somme

Risques mitiges par le systeme

Defaut de paiement (alertes preventives)

Renforcement de la solidarite	Absence de cadre legal (contrat virtuel)
Financement de projets personnels	Conflits de gestion (historique immutable)
Transparence totale des flux financiers	Problemes d'identite (verification KYC)
Rappels automatiques des cotisations	Ordre de distribution conteste (defini des le depart)

2. ANALYSE DES BESOINS

2.1 Acteurs du Système

Acteur	Description	Droits principaux
Visiteur	Utilisateur non authentifié qui découvre l'application	Inscription, consultation de la page d'accueil, consultation des tontines ouvertes
Membre	Utilisateur inscrit et ayant rejoint une tontine	Cotiser, consulter son tableau de bord, chat, historique
Créateur / Admin tontine	Membre qui a créé la tontine	Tous droits membres + configuration, validation, gestion des membres, ordre
Super Administrateur	Administrateur système de la plateforme	Gestion globale, supervision, statistiques

2.2 Cas d'Utilisation (Use Cases)

UC-AUTH : Gestion du Compte Utilisateur

Module Authentification & Profil
UC-01 : S'inscrire avec email/téléphone + confirmation OTP
UC-02 : Se connecter (authentification JWT avec refresh token)
UC-03 : Réinitialiser le mot de passe via email
UC-04 : Modifier le profil (photo, nom, contact, mot de passe)
UC-05 : Vérifier son identité (upload carte d'identité pour tontines ouvertes aux inconnus)

UC-TONTINE : Gestion de la Tontine (Createur)

Module Creation & Configuration
UC-06 : Créer une tontine (nom, montant, fréquence, durée, % frais, mode ordre)
UC-07 : Inviter des membres (email, lien de partage, WhatsApp)
UC-08 : Valider les adhérents (vérification identité : email/téléphone ou pièce d'identité)
UC-09 : Définir et modifier l'ordre de distribution (contrainte : non-modifiable après prise)
UC-10 : Lancer officiellement la tontine (après signature du contrat par tous les membres)
UC-11 : Clôre ou reconduire une tontine arrivée à terme

UC-MEMBRE : Participation à la Tontine

Module Participation & Cotisations
UC-12 : Accepter l'invitation à rejoindre une tontine
UC-13 : Lire et signer le contrat virtuel de tontine (case d'acceptation)
UC-14 : Effectuer une cotisation (simulation LiqdiCash/Orange Money)
UC-15 : Consulter son tableau de bord (solde, prochain tour, statut cotisations)

- UC-16 : Consulter l'historique complet des paiements et distributions
- UC-17 : Recevoir des notifications de rappel de cotisation (J-3, J-1)
- UC-18 : Recevoir le versement de la cagnotte (simulation + 0.5% frais automatiquement deduits)
- UC-19 : Participer au chat de groupe de la tontine en temps reel (WebSocket)

2.3 Regles Metier Fondamentales

1. Calcul automatique du nombre de membres : $\text{NombreMembres} = \text{DureeTotale} / \text{FrequenceCotisation}$.
2. Une fois qu'un membre a reçu la cagnotte (aRecu = true), son ordre de distribution ne peut plus être modifié.
3. Le pourcentage de frais administratifs (ex. 0.5%) est déduit automatiquement du montant reçu par le bénéficiaire.
4. Un membre qui n'a pas cotisé à son tour peut tout de même recevoir son tour - le système ne bloque pas la distribution mais signale le déficit.
5. La tontine ne peut démarrer qu'après signature du contrat par TOUS les membres sans exception.
6. Pour les tontines ouvertes aux inconnus, la vérification d'identité (pièce d'identité image) est obligatoire avant validation.
7. Les paiements sont simulés via l'interface LiqdiCash/Orange Money (mode simulation pour environnement scolaire).
8. L'ordre de distribution peut être : automatique (tirage aléatoire au lancement) ou manuel (défini par le créateur).

3. CONCEPTION

3.1 Diagramme de Classes - Entites Principales

Entite	Attributs Cles	Relations
User	id, nom, prenom, email, telephone, motDePasseHash, photo, statut, dateInscription, roleSysteme, estVerifie, urlCnib	HAS MANY Tontine (creees), BELONGS TO MANY Tontine (via Participation)
Tontine	id, nom, montantCotisation, frequence (ENUM), dureeTotale, nbMembresAttendu, dateDebut, dateFin, statut (ENUM), pourcentageFrais, creatorId	HAS MANY Participation, HAS MANY Cotisation, HAS ONE Contrat, HAS MANY Message
Participation	id, userId, tontineId, ordreDistribution, aRecu (bool), dateAdhesion, aSigneContrat, statutVerifIdentite, pieceIdentiteUrl	BELONGS TO User, BELONGS TO Tontine, HAS MANY Cotisation
Cotisation	id, participationId, tontineId, montant, datePrevue, datePaiement, statut (ENUM), simulationRef, cycleNumero	BELONGS TO Participation
Distribution	id, tontineId, beneficiaireId, montantBrut, montantFrais, montantNet, datePrevue, dateEffective, cycleNumero, statut	BELONGS TO Tontine, BELONGS TO User
Contrat	id, tontineId, texteContrat, dateCreation	BELONGS TO Tontine, HAS MANY SignatureContrat
SignatureContrat	id, contratId, userId, dateSignature, ipAddress, accepte (bool)	BELONGS TO Contrat, BELONGS TO User
Message	id, tontineId, senderId, contenu, dateEnvoi, typeMessage (text/system)	BELONGS TO Tontine, BELONGS TO User (sender)
Notification	id, userId, type, titre, contenu, estLue, dateCreation, lienAction	BELONGS TO User
Invitation	id, tontineId, emailInvite, token (unique UUID), statut (ENUM), dateExpiration	BELONGS TO Tontine

--	--	--

Enumerations (Enums)

Enum	Valeurs possibles
FrequenceCotisation	QUOTIDIEN HEBDOMADAIRE MENSUEL PERSONNALISE
StatutTontine	CONFIGURATION EN_ATTENTE_SIGNATURES ACTIF SUSPENDU TERMINE
StatutCotisation	EN_ATTENTE PAYE EN_RETARD MANQUANT
StatutDistribution	PLANIFIEE EFFECTUEE REPORTEE
StatutVerifIdentite	NON_REQUIS EN_ATTENTE VALIDE REJETE
StatutInvitation	PENDING ACCEPTE EXPIRE REFUSE

3.2 Diagrammes de Sequence

3.2.1 Sequence : Creation et Demarrage d'une Tontine

Scenario complet de creation jusqu'au lancement

1. Createur -> Frontend : Renseigne les parametres (nom, montant, frequence, duree, % frais, ordre)
2. Frontend -> API POST /tontines : Envoie les donnees de configuration
3. API -> Service Tontine : Valide et calcule NombreMembresAttendu = Duree / Frequence
4. Service Tontine -> DB : Cree l'entite Tontine en statut CONFIGURATION
5. Service Tontine -> Service Contrat : Genere automatiquement le texte du contrat
6. API -> Frontend : Retourne tontineId + lien d'invitation unique (UUID token)
7. Createur -> Frontend : Partage le lien (email / WhatsApp / copie directe)
8. Membres -> Frontend : Cliquent sur le lien, creent un compte si necessaire
9. Membres -> API POST /invitations/{token}/accepter : Rejoint la tontine
10. API -> Service Verification : Si tontine ouverte => demande upload piece d'identite
11. Createur -> API PATCH /tontines/{id}/members/{userId}/valider : Valide chaque adherent
12. Tous membres -> API POST /contracts/{tontineId}/sign : Lisent et acceptent le contrat
13. Service Tontine : Surveillance -> tous_membres_signes ? -> passe statut EN_ATTENTE_SIGNATURES -> ACTIF
14. Service Scheduler : Planifie automatiquement tous les cycles (dates cotisations + distributions)
15. Service Notification : Envoie notifications de demarrage a TOUS les membres

3.2.2 Sequence : Cotisation d'un Membre

Scenario : Un membre effectue sa cotisation

1. Service CRON : J-3 avant la date limite -> envoi rappel email/notification a tous les membres non-payés
2. Service CRON : J-1 avant la date limite -> alerte spéciale si cotisation toujours manquante (signal rouge)
3. Membre -> Frontend : Clique sur 'Payer ma cotisation'
4. Frontend -> API POST /tontines/{id}/contributions : Declenche le paiement simulé
5. API -> Service Paiement Simulation : Simule la transaction LiquiCash/Orange Money
6. Service Paiement -> DB : Cree Cotisation statut PAYE + simulationRef unique
7. Service Notification -> Createur : Notifie qu'un membre a payé
8. API -> Frontend : Retourne confirmation + reçu de paiement avec détails
9. Service Tontine : Verifie si TOUS les membres actifs du cycle ont cotisé
10. Si tous cotisent -> planifie/declenche la distribution pour le beneficiaire du cycle

3.2.3 Sequence : Distribution de la Cagnotte

Scenario : Versement automatique au beneficiaire du cycle

1. Service CRON : Date de distribution atteinte -> declenche automatiquement le processus
2. Service Distribution -> DB : Recupere le beneficiaire du cycle selon l'ordre defini
3. Service Distribution : Calcule montantBrut = NombreMembres x MontantCotisation
4. Service Distribution : Calcule montantFrais = montantBrut x pourcentageFrais / 100
5. Service Distribution : Calcule montantNet = montantBrut - montantFrais
6. Service Paiement Simulation : Simule le virement vers le compte du beneficiaire
7. Service Distribution -> DB : Cree Distribution statut EFFECTUEE, met aRecu=true pour ce membre
8. Service Notification -> Beneficiaire : 'Vous avez reçu X FCFA (frais Y FCFA deduits)'
9. Service Notification -> Tous membres : Informe du versement effectue
10. Service Tontine : Verifie si c'etait le dernier cycle -> si oui passe statut TERMINE + notification de cloture

3.2.4 Sequence : Chat en Temps Reel (WebSocket)

Scenario : Communication en temps reel entre membres

1. Membre -> Frontend : Ouvre l'onglet chat de sa tontine
2. Frontend -> Socket.io : etablit connexion ws://[host] avec token JWT dans le handshake
3. Socket.io Server -> Middleware Auth : Verifie le JWT, identifie l'utilisateur
4. Socket.io Server -> DB : Verifie que l'utilisateur est bien membre de cette tontine
5. Socket.io Server : Ajoute le client dans la 'room' identifiee par tontineId
6. Frontend -> Socket.io (emit 'load_history') : Charge les derniers messages
7. Membre -> Frontend : Tape et envoie un message
8. Frontend -> Socket.io (emit 'send_message') : Envoie {tontineId, contenu}
9. Socket.io Server -> DB : Persiste le message avec senderId, tontineId, dateEnvoi
10. Socket.io Server (broadcast to room) : Diffuse le message a TOUS les membres connectes

11. Service Notification : Pour membres deconnectes -> envoie notification push/email

3.3 Diagramme d'Activite - Cycle de Vie d'une Tontine

9. CREATION : Le createur configure les parametres -> le systeme calcule NombreMembresAttendu
10. INVITATION : Envoi des invitations (email/lien/WhatsApp) -> les membres rejoignent la tontine
11. VALIDATION : Le createur valide chaque adherent (identite verifiee si tontine ouverte)
12. SIGNATURE : Tous les membres lisent et signent le contrat virtuel numeriquement
13. DEMARRAGE : Statut passe a ACTIF -> planification automatique de tous les cycles
14. CYCLE ACTIF : CRON rappels -> cotisations membres -> verification -> distribution -> cycle suivant
15. SUPERVISION : Signalement des cotisations en retard, alertes, tableau de bord temps reel
16. CLOTURE : Dernier cycle termine -> statut TERMINE -> proposition de reconduction

4. DEVELOPPEMENT

4.1 Stack Technique

4.1.1 Backend

Technologie	Role	Justification
Node.js + Express.js	Serveur API REST	Non-bloquant, ideal pour I/O frequentes, vaste ecosysteme
Socket.io	Chat temps reel (WebSocket)	Gestion native des rooms, fallback automatique, facile avec Express
PostgreSQL	Base de donnees relationnelle	ACID, excellent pour transactions financieres, relations complexes
Prisma ORM	Acces BDD + migrations	Schema-first, type safety, migrations automatiques, seeding
JWT + bcrypt	Auth & securite	Standard industrie : JWT stateless + hachage securise des mots de passe
Nodemailer	Envoi emails	Rappels, invitations, confirmations, contrats par email
node-cron	Taches planifiees	Rappels automatiques J-3/J-1, declenchement distributions a heure fixe
Multer	Upload fichiers	Gestion uploads pieces d'identite (restriction taille + type MIME)
Joi / Zod	Validation donnees	Validation robuste de toutes les entrees avant traitement backend
Swagger (OpenAPI)	Documentation API	Documentation interactive, contrat API pour le front, mock server

4.1.2 Frontend

Technologie	Role	Justification
React.js + Vite	Framework UI + bundler	Composants reactifs, HMR instantane, adapte aux dashboards dynamiques
TailwindCSS	Styling	Utility-first, design responsive rapide, coherence visuelle garantie
React Router v6	Navigation SPA	Routing declaratif, routes protegees, gestion des redirections
Axios	Client HTTP	Intercepteurs JWT, gestion centralisee des erreurs, instances configurables
Socket.io-client	WebSocket client	Integration seamless avec Socket.io serveur, gestion reconnexion
React Query (TanStack)	Cache & etat serveur	Cache automatique, refetch intelligent, optimistic updates
Chart.js / Recharts	Graphiques	Visualisation cotisations, dashboards financiers, courbes historique
React Hook Form	Formulaires	Performance, validation, gestion etats formulaires complexes

4.2 Architecture du Projet

4.2.1 Structure Backend

Organisation des repertoires - API Backend Node.js

src/

config/ -> database.js, jwt.js, mailer.js, socket.js, env.js
 middlewares/ -> auth.middleware.js, errorHandler.js, upload.js, rateLimiter.js
 modules/
 auth/ -> auth.router.js | auth.controller.js | auth.service.js
 users/ -> users.router.js | users.controller.js | users.service.js
 tontines/ -> tontines.router.js | tontines.controller.js | tontines.service.js
 contributions/-> contributions.router.js | contributions.service.js
 distributions/-> distributions.router.js | distributions.service.js
 chat/ -> chat.gateway.js (gestion WebSocket / Socket.io rooms)
 contracts/ -> contracts.service.js (generation texte + gestion signatures)
 notifications/-> notifications.service.js | notification.events.js
 invitations/ -> invitations.router.js | invitations.service.js
 jobs/ -> cron.scheduler.js (rappels J-3, J-1, declenchement distributions)
 prisma/ -> schema.prisma | migrations/ | seed.js
 server.js -> point d'entree Express + Socket.io + CRON

4.2.2 Structure Frontend

Organisation des repertoires - Application React

src/

assets/ -> images, icones, fonts
 components/ -> Button, Modal, Badge, Card, Avatar, NotificationBell, ChatBubble...
 hooks/ -> useAuth.js | useTontine.js | useChat.js | useNotifications.js
 pages/
 auth/ -> Login.jsx | Register.jsx | ForgotPassword.jsx
 dashboard/ -> Dashboard.jsx | MesTontines.jsx
 tontine/ -> Creer.jsx | Detail.jsx | Membres.jsx | Cotisations.jsx
 -> Distribution.jsx | Chat.jsx | Contrat.jsx | Ordre.jsx
 profile/ -> Profil.jsx | VerificationIdentite.jsx
 admin/ -> SuperAdmin.jsx (supervision globale)
 services/ -> api.js (instance Axios) | socket.js | auth.service.js
 store/ -> authStore.js | notificationStore.js (Zustand ou Context API)
 routes/ -> AppRouter.jsx | ProtectedRoute.jsx
 utils/ -> formatCurrency.js | dateHelpers.js | validators.js
 App.jsx + main.jsx

4.3 Endpoints API Reference

4.3.1 Authentification & Utilisateurs

Methode	Endpoint	Description	Auth
POST	/api/auth/register	Inscription utilisateur + envoi OTP	Non
POST	/api/auth/login	Connexion, retourne JWT + refresh token	Non
POST	/api/auth/verify-otp	Vérification email/telephone par OTP	Non
POST	/api/auth/refresh-token	Renouveler le token JWT expire	Oui
POST	/api/auth/forgot-password	Demander reinitialisation MDP	Non
PATCH	/api/users/profile	Modifier le profil utilisateur	Oui
POST	/api/users/verify-identity	Upload piece d'identite	Oui

4.3.2 Tontines & Membres

Methode	Endpoint	Description	Auth
GET	/api/tontines	Liste des tontines de l'utilisateur connecte	Oui
POST	/api/tontines	Creer une nouvelle tontine	Oui
GET	/api/tontines/:id	Detail complet d'une tontine	Membre
PATCH	/api/tontines/:id	Modifier config (avant demarrage uniquement)	Createur
POST	/api/tontines/:id/start	Lancer la tontine (toutes signatures ok)	Createur
GET	/api/tontines/:id/members	Liste membres + statuts cotisations	Membre
PATCH	/api/tontines/:id/members/:uid/validate	Valider un adherent	Createur
PATCH	/api/tontines/:id/order	Modifier ordre (contrainte aRecu=false)	Createur

4.3.3 Cotisations, Distributions, Contrats, Chat & Notifications

Methode	Endpoint	Description
GET	/api/tontines/:id/contributions	Historique cotisations du cycle en cours
POST	/api/tontines/:id/contributions	Simuler le paiement d'une cotisation
GET	/api/tontines/:id/distributions	Historique complet des distributions
GET	/api/contracts/:tontineId	Recuperer le contrat de la tontine
POST	/api/contracts/:tontineId/sign	Signer numeriquement le contrat
POST	/api/invitations/send	Envoyer invitations (email/lien/WhatsApp)
POST	/api/invitations/:token/accept	Accepter une invitation via lien token
GET	/api/tontines/:id/messages	Historique du chat de la tontine
GET	/api/notifications	Toutes les notifications de l'utilisateur

PATCH	/api/notifications/:id/read	Marquer une notification comme lue
WS	ws://[host] (Socket.io)	Chat temps reel : events send_message, load_history, user_joined

4.4 Decoupage des Equipes & Coordination

Equipe	Modules / Pages Responsables	Livrables
Backend Equipe 1	Auth, Users, Tontines (CRUD), Invitations, Contrats, Verification identite	Endpoints /auth/*, /users/*, /tontines/* (config), /invitations/*, /contracts/*
Backend Equipe 2	Contributions, Distributions, Notifications, Scheduler CRON, Chat WebSocket	Endpoints /contributions/*, /distributions/*, /notifications/*, Socket.io gateway
Frontend Equipe 1	Auth (Login/Register), Dashboard, Creation Tontine, Gestion Membres, Profil, Verification identite	Pages /login, /register, /dashboard, /tontines/create, /tontines/:id/members, /profile
Frontend Equipe 2	Cotisations, Distributions, Chat temps reel, Notifications, Historique, Signature Contrat	Pages /tontines/:id/contributions, /chat, /contract, composant NotificationCenter

Strategie de developpement parallele (Backend & Frontend)

1. API FIRST : Le backend genere une documentation Swagger/OpenAPI complete avant tout developpement frontend.
2. MOCK SERVER : Le frontend utilise json-server ou MSW (Mock Service Worker) pour simuler les reponses API.
3. ENV VARIABLE : VITE_API_URL bascule entre le mock local et le vrai backend (staging/production).
4. TYPES PARTAGES : Un fichier types.ts commun (ou doc JSON Schema) definit les structures de donnees echangees.
5. SPRINTS D'INTEGRATION : Session d'integration frontend/backend a chaque fin de sprint pour valider la compatibilite.
6. GIT FLOW : Branches feature/backend-equipe1, feature/backend-equipe2, feature/frontend-equipe1, feature/frontend-equipe2.
7. TESTS : Tests unitaires backend (Jest), tests d'integration (Supertest), tests composants frontend (React Testing Library).

4.5 Securite & Bonnes Pratiques

- Authentification JWT : access token 15min + refresh token 7 jours, rotation securisee
- Mots de passe : hachage bcrypt (saltRounds=10), jamais stockes en clair
- Validation : toutes les entrees validees avec Joi/Zod AVANT traitement (whitelist des champs)
- CORS : configure pour n'autoriser QUE le domaine frontend connu
- Rate limiting : protection sur /auth/login (5 tentatives/15min), /invitations (10/heure)
- Uploads : noms de fichiers aleatoires (UUID), restriction type MIME (JPEG/PNG uniquement), taille max 5MB
- Autorisation : chaque route verifie que l'utilisateur est bien membre de la tontine concerne

-
- Transactions PostgreSQL : cotisations et distributions executees en transactions atomiques
 - HTTPS obligatoire en production, variables sensibles dans .env (jamais committees)

5. PLANIFICATION & LIVRABLES

5.1 Phases du Projet

Phase	Duree	Activites principales	Livrable
Phase 1 - Setup & Architecture	1 semaine	Init projets, config DB Prisma, Swagger, mocks MSW, structure de base, CI/CD basique	Squelette backend + frontend + doc API
Phase 2 - Auth & Users	1 semaine	Inscription, connexion JWT, OTP, profil, verification identite, middleware auth	Module authentication complet
Phase 3 - Tontines & Membres	2 semaines	CRUD tontines, invitations (email/WhatsApp), validation membres, contrat virtuel, ordre	Creation et demarrage d'une tontine
Phase 4 - Cotisations & Distributions	2 semaines	Simulation paiements, CRON rappels J-3/J-1, distributions automatiques, alertes	Cycle de tontine fonctionnel E2E
Phase 5 - Chat & Notifications	1 semaine	WebSocket Socket.io, messagerie temps reel, centre de notifications in-app	Chat + notifications operationnels
Phase 6 - Tests & Demo	1 semaine	Tests d'integration, correction bugs, polish UI, demo finale, documentation	Application livree et documentee

5.2 Recapitulatif des Fonctionnalites par Priorite

Fonctionnalite	Priorite	Equipe(s)
Inscription / Connexion / OTP / JWT	P0 - Critique	BE1 + FE1
Creation et configuration d'une tontine	P0 - Critique	BE1 + FE1
Invitations membres (email, lien, WhatsApp)	P0 - Critique	BE1 + FE1
Validation adherents + verification identite KYC	P0 - Critique	BE1 + FE1
Contrat virtuel + signature numerique	P0 - Critique	BE1 + FE2
Simulation cotisations LiqdiCash/Orange	P0 - Critique	BE2 + FE2
Distribution automatique de la cagnotte	P0 - Critique	BE2 + FE2
Calcul et deduction des frais administratifs	P1 - Important	BE2
Rappels automatiques CRON (J-3, J-1)	P1 - Important	BE2
Gestion ordre distribution (contrainte aRecu)	P1 - Important	BE1 + FE1
Chat temps reel WebSocket (Socket.io)	P1 - Important	BE2 + FE2
Centre de notifications in-app	P1 - Important	BE2 + FE2
Tableau de bord et historique complet	P1 - Important	FE1 + FE2
Signal cotisation manquante a J-1	P1 - Important	BE2 + FE2
Super Admin - supervision globale	P2 - Optionnel	BE1 + FE1

5.3 Conclusion

Ce cahier des charges definit de maniere exhaustive le systeme de gestion des tontines, en couvrant les aspects fonctionnels, techniques, architecturaux et organisationnels. Le decoupage modulaire du projet garantit une collaboration efficace entre les quatre equipes de developpement, tandis que la strategie API-first et l'utilisation de mocks permettent un developpement en parallele sans dependance bloquante.

La plateforme proposee repond directement aux problematiques identifiees dans la gestion traditionnelle des tontines : manque de transparence, absence de tracabilite, difficultes de communication et risques lies a l'identite des membres. En combinant automatisation intelligente, securite des donnees et facilite d'utilisation, ce systeme vise a moderniser et democratiser la pratique de la tontine.

Points forts du systeme - Resume

Calcul automatique du nombre de membres selon la frequence et la duree du cycle

Contrat virtuel signe par tous avant le demarrage : cadre legal numerique garantissant l'engagement

Verification d'identite KYC pour les tontines avec inconnus (upload piece d'identite image)

Ordre de distribution protege apres prise (aRecu=true) : integrite et equite garanties

Simulation paiement LiqdiCash/Orange Money : facilement remplaçable par vraie API en production

Chat WebSocket integre : communication en temps reel au sein du groupe de tontine

Rappels automatiques CRON J-3 et J-1 : ponctualite des cotisations assuree

Architecture 4 equipes decouplee avec API-first : developpement parallele sans blocage

Systeme non-bloquant : un membre peut recevoir son tour meme sans avoir cotise (signal uniquement)